

软件工程 II 简答题概念速记

面向前三题概念简答：依据今年课件、123.txt 答疑、语音版重点、考试题型整理，并参考 2020 试卷问法风格。

使用方法：每个概念按“可能问法 → 标准答法 → 记忆点 → 易错点”记。考试答简答题不要只背名词，要写出：定义、作用/意义、典型例子或产物、和相近概念的区别。

2020 试卷前三题给出的问法风格

题目	考查类型	今年复习启发
什么是软件工程？	基础定义	概念题可能直接问定义，要能用规范表述解释“工程化开发软件”。
持续集成的好处和相关工具链。	工程实践/工具链	往年出现过的工程活动不能忽略，要会答好处、流程和典型工具。
简述软件生命周期含义，并列举一种软件生命周期模型。	过程模型	生命周期、过程模型、阶段产物这类概念仍要准备。

一、软件工程基础与过程

可能问法	什么是软件工程？
标准答法	软件工程是将系统化、规范化、可度量的方法应用于软件的开发、运行和维护，并研究这些方法的学科。它强调用工程化方式管理软件复杂性，使软件在质量、成本、进度和可维护性之间取得平衡。
记忆点	系统化 + 规范化 + 可度量；开发 + 运行 + 维护。
易错点	不要只答“写代码的方法”。软件工程覆盖需求、设计、构造、测试、交付、维护和管理。
可能问法	软件工程要解决什么核心问题？
标准答法	核心是解决软件复杂性、变化性、不可见性和协作困难。通过抽象、分解、模块化、信息隐藏、过程管理和质量保证，使软件可理解、可修改、可验证、可交付。
记忆点	复杂、变化、不可见、协作。
易错点	不要把软件工程理解成文档工程；它的目标是提升软件质量和开发效率。
可能问法	什么是软件生命周期？
标准答法	软件生命周期是软件从概念提出、需求分析、设计、构造、测试、交付、运行维护直到退役的全过程。生命周期模型则是组织这些阶段及其关系的过程模型。
记忆点	从出生到退役；模型是阶段组织方式。
易错点	生命周期是“过程范围”，瀑布/迭代/敏捷是“模型”。
可能问法	列举一种软件生命周期模型并说明特点。
标准答法	瀑布模型按需求、设计、实现、测试、运行维护等阶段顺序推进，阶段边界清晰、文档完整，适合需求较稳定、变更较少、对过程控制要求高的项目。缺点是反馈晚、适应变化能力弱。
记忆点	瀑布：顺序、文档、稳定需求。也可答迭代、增量、敏捷。
易错点	只列名称不够，要写适用场景和优缺点。
可能问法	什么是迭代/增量开发？
标准答法	迭代开发强调多轮循环，每轮都分析、设计、实现、测试并反馈；增量开发强调每次交付系统的一部分功能。二者常结合使用，通过小步交付降低风险并适应变化。
记忆点	迭代是反复改进，增量是逐步交付。
易错点	不要把迭代简单理解为“返工”；迭代是有计划的反馈循环。
可能问法	什么是持续集成？它有什么好处？
标准答法	持续集成是开发者频繁将代码合并到主干或共享分支，并通过自动构建、自动测试、静态检查等流程及时发现集成问题的实践。好处包括尽早发现缺陷、降低集成风险、保持软件可运行、提高反馈速度和团队协作效率。
记忆点	频繁合并 + 自动构建 + 自动测试 + 快速反馈。
易错点	相关工具链可答 Git、GitHub/GitLab、Maven/Gradle、Jenkins/GitHub Actions、JUnit、SonarQube、Docker 等。

可能问法	什么是软件过程？
标准答法	软件过程是为了开发和维护软件而执行的一组活动、方法、实践和产物组织方式。典型活动包括需求、设计、构造、测试、交付和维护。
记忆点	过程 = 活动 + 方法 + 产物 + 顺序/组织。
易错点	过程不是固定模板，不同项目会选择不同生命周期模型和实践。

可能问法	什么是软件配置管理？
标准答法	软件配置管理是识别、控制、记录和审计软件配置项变化的活动，目标是保证版本可追踪、变更可控制、构建可复现。常见实践包括版本控制、分支管理、变更控制、发布标记和构建管理。
记忆点	管版本、管变更、保可追踪。
易错点	不要只答 Git；Git 是工具，配置管理是工程活动。

二、项目管理与工程管理概念

可能问法	什么是项目管理？
标准答法	项目管理是在有限资源、时间和质量约束下，对范围、进度、成本、风险、人员和沟通进行计划、组织、跟踪和控制，以达成项目目标的活动。
记忆点	范围、进度、成本、质量、风险、人。
易错点	往年可能考基础管理概念，答题时写“约束”和“控制”。

可能问法	什么是风险管理？
标准答法	风险管理是识别可能影响项目目标的不确定事件，评估其概率和影响，并制定规避、缓解、转移或接受策略的过程。软件项目常见风险包括需求变更、技术不确定、进度延期、人员流动和质量问题。
记忆点	识别 → 评估 → 应对 → 跟踪。
易错点	风险不是已经发生的问题，而是可能发生的不确定事件。

可能问法	为什么工程管理对软件开发重要？
标准答法	软件开发通常需要多人协作，且需求、技术和环境会变化。工程管理通过计划、分工、沟通、跟踪和风险控制降低协作成本，帮助团队在质量、进度和成本之间达成平衡。
记忆点	多人协作 + 不确定性 + 约束平衡。
易错点	不要只从“管人”角度答，管理还包括计划、过程、质量和风险。

三、需求工程概念

可能问法	什么是需求工程？
标准答法	需求工程是围绕需求获取、分析、建模、规格说明、验证和管理的一组活动，目标是明确系统为什么建、为谁建、要解决什么问题以及系统应满足哪些功能和质量要求。
记忆点	获取、分析、建模、文档化、验证、管理。
易错点	需求工程不是只写需求文档，而是持续澄清和管理需求。

可能问法	为什么要先明确业务目标？
标准答法	业务目标说明组织为什么要建设系统，是需求分析的出发点和验收追溯源头。明确业务目标可以界定系统范围、凝练成功指标、判断用户需求是否有价值，并避免把技术实现或局部功能误当成真正需求。
记忆点	业务目标 = 为什么建 + 成功指标 + 范围边界。
易错点	业务目标应尽量可度量，如效率提升、成本降低、收益增加。

可能问法	业务需求、用户需求、系统级需求有什么区别？
标准答法	业务需求描述组织或客户希望达成的高层业务目标；用户需求描述用户借助系统完成的任务或目标；系统级需求从系统角度精确描述系统与外部实体的交互、输入输出、状态变化和约束。三者从抽象到具体、从目标到规格逐层展开。
记忆点	业务：为什么；用户：用户要做什么；系统级：系统如何对外交互。
易错点	不要把系统级需求写成数据库表或代码实现。

可能问法	功能需求与非功能需求有什么区别？
标准答法	功能需求描述系统应提供的功能或行为，即系统做什么；非功能需求描述系统在质量属性、约束和运行条件方面的要求，即系统做得怎样。非功能需求包括性能、可靠性、可用性、安全性、可维护性、易用性等。
记忆点	功能 = 做什么；非功能 = 做得怎样。
易错点	非功能需求会强烈影响架构设计，例如安全性、可维护性、性能。
可能问法	什么是需求获取？
标准答法	需求获取是通过访谈、观察、问卷、原型、文档分析、头脑风暴等方式，从用户、客户、领域专家和现有系统中发现业务问题、用户任务、约束和期望的活动。
记忆点	目标是发现问题和需求，不是直接给方案。
易错点	用户能感受到问题，但不一定能提出正确的软件方案。
可能问法	什么是用例？
标准答法	用例描述参与者为了达成某个有价值的业务目标而与系统进行的一组交互。一个用例是多个场景的集合，包括正常流程和扩展流程。
记忆点	用例 = 用户目标 + 系统交互 + 场景集合。
易错点	登录、增删改查、连接数据库通常不是独立用例，可能只是步骤或技术细节。
可能问法	用例图有哪些组成部分？
标准答法	用例图主要包含参与者、用例、系统边界和关系。关系包括关联、包含 include、扩展 extend、泛化 generalization。它用于集中展示参与者和系统提供的用户目标。
记忆点	Actor、Use Case、Boundary、Relation。
易错点	include 表示必然包含的公共行为；extend 表示可选或条件发生的扩展行为。
可能问法	用例描述通常包含哪些内容？
标准答法	完整用例描述通常包括 ID、名称、参与者、触发条件、前置条件、后置条件、正常流程、扩展流程、特殊需求和补充说明。考试重点是能写清正常流程与扩展流程。
记忆点	前 6 项是上下文，后 4 项是行为和约束。
易错点	前置条件不是用例步骤，例如“已登录”通常是前置条件，不写进正常流程。
可能问法	什么是场景？场景和用例有什么关系？
标准答法	场景是用例中的一条具体执行路径，可以是主成功场景，也可以是异常或替代场景。用例是所有相关场景的集合。
记忆点	场景是一条路径，用例是一组路径。
易错点	不要把一个扩展场景当成新的完整用例，除非它本身有独立业务目标。

可能问法	什么是用户故事？
标准答法	用户故事是以自然语言描述用户价值的小粒度需求，常用格式是：作为某类用户，我希望完成某功能，以便获得某价值。好的用户故事应符合 INVEST：独立、可讨论、有价值、可估计、小、可测试。
记忆点	角色 + 功能 + 价值。
易错点	用户故事不是合同，细节可在讨论和验收标准中补充。
可能问法	什么是概念类图？
标准答法	概念类图也称领域模型，用于描述问题域中的重要概念、属性、关联和多重性，捕获系统要理解的静态业务结构。它属于分析阶段，通常不包含方法。
记忆点	概念类图：类 + 属性 + 关联 + 多重性；无方法。
易错点	不要把技术类、DAO、Controller、数据库操作放进概念类图。
可能问法	系统顺序图和详细顺序图有什么区别？
标准答法	系统顺序图处于需求分析阶段，把系统看成黑盒，描述参与者与系统之间的消息顺序；详细顺序图处于设计阶段，展开系统内部对象或类之间的协作，如 Controller、Service、DAO、实体对象之间的消息。
记忆点	系统顺序图：Actor ↔ System；详细顺序图：对象 ↔ 对象。
易错点	顺序图是从行为/交互角度建模，不是状态角度。
可能问法	什么是状态图？绘制状态图的关键是什么？
标准答法	状态图描述某个主体在生命周期中的状态及事件触发下的状态转换。绘制关键是先明确主体是谁，再识别稳定状态、触发事件、守卫条件和动作。
记忆点	主体 → 状态 → 事件 → 条件/动作。
易错点	主体不同状态图不同，不要把多个主体混在一张图里。
可能问法	需求规格说明书为什么仍有意义？
标准答法	即使考试不要求完整书写传统需求规格说明书，规格说明所承载的系统交互细节仍然重要。它通过结构化文本、用例、用户故事、操作契约、注释或其他方式减少歧义，使开发和测试有共同依据。
记忆点	重点不是格式，而是精确表达交互细节。
易错点	不写传统 SRS 不等于不表达系统级需求。

四、体系结构与设计基础

可能问法	什么是软件设计？
标准答法	软件设计是在需求和约束基础上，确定软件的结构、组件、接口、数据和行为实现方案的活动。它把“要做什么”转化为“如何组织软件来实现”。
记忆点	需求到实现之间的桥梁。
易错点	设计不只是画图，还包括职责分配、接口定义和设计理由。

可能问法	什么是软件体系结构？
标准答法	软件体系结构是系统的高层结构，包括主要构件、构件之间的连接关系、接口、约束以及指导系统演化的设计决策。它关注系统整体组织和关键质量属性。
记忆点	构件 + 连接 + 接口 + 约束 + 决策。
易错点	体系结构不是某个框架名称，也不是代码目录截图。

可能问法	什么是体系结构风格？
标准答法	体系结构风格是一类系统组织方式的抽象，规定了组件类型、连接器类型和拓扑约束。常见风格包括主程序-子程序、面向对象、分层、MVC、管道-过滤器、客户端-服务器、微服务等。
记忆点	风格 = 组件词汇 + 连接器 + 约束。
易错点	风格选择应由需求、质量属性和项目约束驱动。

可能问法	什么是分层体系结构？
标准答法	分层体系结构把系统按职责划分为若干层，上层通过接口使用下层服务，下层隐藏实现细节。典型三层为展示层、业务逻辑层和数据访问层。优点是职责清晰、降低耦合、便于维护和团队分工。
记忆点	展示层、业务逻辑层、数据访问层。
易错点	逻辑分层不等于物理分离，单体系统也可以有清晰逻辑分层。

可能问法	什么是 MVC？
标准答法	MVC 是 Model-View-Controller。Model 管理数据和业务逻辑，View 负责展示，Controller 处理用户输入、更新模型并选择视图。它通过分离展示、输入控制和业务数据逻辑来降低耦合。
记忆点	Model 管业务，View 管展示，Controller 管输入协调。
易错点	MVC 可与分层结合，Web 系统中 View 和 Controller 常位于展示层。

可能问法	什么是 4+1 视图？
标准答法	4+1 视图从多个视角描述体系结构：逻辑视图关注功能划分和类/包组织，开发视图关注代码模块，进程视图关注并发和运行时交互，物理视图关注部署拓扑，场景视图用关键用例验证其他视图。
记忆点	逻辑、开发、进程、物理 + 场景。
易错点	场景视图是粘合剂，用例能否穿透各视图可检验架构一致性。

可能问法	包图用于表达什么？由哪些部分组成？
标准答法	包图用于表示系统包、层、模块及其依赖关系。组成部分包括包、包内类或接口、依赖箭头、接口、分层结构以及 VO/PO 等数据对象。课程中常用包图表达体系结构逻辑视图。
记忆点	包 + 依赖 + 接口 + 分层 + PO/VO。
易错点	包图不是类图细节大全，重点是模块边界和依赖方向。

可能问法	PO 和 VO 有什么区别？
标准答法	PO 是 Persistent Object，面向持久化，通常对应数据库记录，用于数据层与业务层交互；VO 是 Value Object，面向展示层数据传递，只包含前端或上层需要的数据，用于展示层与业务层交互。
记忆点	PO 对数据库，VO 对展示。
易错点	VO 的意义是减少不必要数据暴露，降低耦合。

可能问法	DAO 是什么？
标准答法	DAO 是 Data Access Object，数据访问对象，位于数据访问层，负责封装数据库或持久化存储的访问细节，为业务逻辑层提供数据查询、保存、更新、删除等接口。
记忆点	DAO 封装数据访问，业务层不直接写 SQL。
易错点	DAO 是工程设计角色，不等同于数据库表本身。

可能问法	供接口和需接口是什么意思？
标准答法	供接口表示模块对外提供的服务；需接口表示模块为了完成职责需要调用外部模块提供的服务。完整的体系结构接口不仅包括提供什么，也包括需要什么。
记忆点	提供给别人 = 供接口；自己依赖别人 = 需接口。
易错点	不要把体系结构接口局限为 Java interface 语法，它是更广义的模块契约。

可能问法	什么是架构重要需求 ASR？
标准答法	ASR 是 Architecturally Significant Requirement，指会显著影响体系结构决策的需求，如高并发、安全性、可用性、技术约束、部署约束等。并非所有功能需求都是 ASR。
记忆点	影响架构决策的需求。
易错点	按钮颜色通常不是 ASR；1000 并发、支付加密、7x24 可用可能是 ASR。

五、详细设计、模块化与设计原则

可能问法	什么是详细设计？
标准答法	详细设计是在体系结构约束下，把模块职责进一步细化为类、接口、方法、对象协作、状态和算法的设计活动。它的输出包括设计类图、详细顺序图、状态图、伪代码、接口说明和设计理由。
记忆点	架构到代码之间的类粒度设计。
易错点	详细设计不是随便画几张图，而是可指导构造的实现蓝图。

可能问法	什么是职责和协作？
标准答法	职责是类或对象应维护的数据或应执行的任务；协作是多个对象通过消息传递共同完成较大职责的方式。面向对象详细设计就是合理分配职责并设计协作。
记忆点	职责 = 做什么/维护什么；协作 = 怎么一起做。
易错点	一个对象不应做所有事，应通过委托和协作降低复杂度。

可能问法	GRASP 是什么？
标准答法	GRASP 是通用职责分配软件模式，用于指导面向对象设计中如何把职责分配给类。课程重点包括低耦合、高内聚、信息专家、创建者和控制器。
记忆点	GRASP 不是 GoF 设计模式，而是职责分配原则。
易错点	答题时要说明为什么某职责应给某类，而不是只写模式名。

可能问法	什么是信息专家 Information Expert？
标准答法	信息专家原则认为，应把职责分配给拥有完成该职责所需信息的类。例如计算销售总额时，销售对象知道销售明细，明细知道数量和商品，商品知道价格，因此职责可沿信息所在对象委托。
记忆点	谁有数据，谁负责。
易错点	不是所有逻辑都放 Controller；Controller 可协调，领域对象应承担自身职责。

可能问法	什么是 Creator 创建者原则？
标准答法	Creator 原则用于决定谁负责创建对象。如果类 B 聚合、包含、记录、密切使用类 A，或拥有创建 A 所需初始化数据，则 B 是创建 A 的候选者。优先由整体或容器创建其组成部分。
记忆点	包含/聚合/记录/密切使用/有初始化数据。
易错点	对象创建也会产生耦合，要选择能保持低耦合高内聚的创建者。

可能问法	什么是 Controller 控制器原则？
标准答法	Controller 原则用于分配处理系统事件的职责。可以由系统整体、业务组织、现实角色或人为用例控制器接收外部事件，并协调领域对象完成任务，从而避免表现层直接操作领域对象。
记忆点	Controller 接系统事件，协调而不包办所有业务。
易错点	控制器过胖会导致低内聚和高耦合。

可能问法	什么是内聚和耦合？
标准答法	内聚衡量模块内部元素围绕同一职责的相关程度，越高越好；耦合衡量模块之间依赖强度，越低越好。好的设计追求高内聚、低耦合，以提高可理解性、可修改性和可复用性。
记忆点	内聚看内部，耦合看外部。
易错点	二者不是口号，分析代码时要指出职责是否集中、依赖是否过强。
可能问法	什么是控制耦合？如何改进？
标准答法	控制耦合是调用方传入 flag、类型码等参数来控制被调用方法内部逻辑，使双方都必须理解同一控制约定。可通过多态、策略模式、拆分方法或职责转移来改进。
记忆点	flag/switch 控制行为 = 控制耦合。
易错点	不是所有参数都是控制耦合，关键看参数是否控制对方内部流程。
可能问法	什么是印记耦合？如何改进？
标准答法	印记耦合是把包含大量信息的数据结构或对象传给方法，但该方法只需要其中一小部分，导致被调用方依赖过多数据。改进方式是只传必要参数、定义更小的 DTO/VO 或接口。
记忆点	传太多、不需要也暴露。
易错点	例如 sendEmail 只需要 email，却传整个 Employee。
可能问法	什么是模块化？
标准答法	模块化是把系统分解为相对独立、职责清晰、接口明确的模块，使开发者可以局部理解、修改、测试和替换系统。好的模块化以职责分配和信息隐藏为核心。
记忆点	分解不是切碎，模块要隐藏设计决策。
易错点	模块化不是文件越多越好，关键是边界合理。
可能问法	什么是信息隐藏？
标准答法	信息隐藏要求每个模块隐藏一个可能变化的设计决策，只通过稳定接口与外部交互。这样当内部实现、数据结构或策略变化时，影响被限制在模块内部。
记忆点	隐藏变化，暴露接口。
易错点	private 字段只是手段，信息隐藏的本质是隔离变化。
可能问法	SOLID 包含哪些原则？
标准答法	SOLID 包括：SRP 单一职责，一个类只有一个变化原因；OCP 开闭原则，对扩展开放、对修改关闭；LSP 里氏替换，子类可替换父类；ISP 接口隔离，客户端不依赖不用的方法；DIP 依赖倒置，高层和低层都依赖抽象。
记忆点	S 单一，O 扩展，L 替换，I 隔离，D 抽象。
易错点	考试不要只写英文缩写，要能结合代码说明违反原因和重构方案。

可能问法	什么是开闭原则 OCP？
标准答法	开闭原则要求软件实体对扩展开放、对修改关闭。也就是说新增变化应尽量通过新增类或实现接口完成，而不是频繁修改已有稳定代码。常用多态、策略模式、工厂方法等支持。
记忆点	新增类型不改旧逻辑。
易错点	大量 if/switch 根据类型分支常是 OCP 违反信号。

可能问法	什么是依赖倒置原则 DIP？
标准答法	依赖倒置要求高层模块不依赖低层具体实现，二者都依赖抽象；抽象不依赖细节，细节依赖抽象。它通过接口和依赖注入降低高层业务逻辑对具体数据库、框架或实现类的耦合。
记忆点	高层和低层都朝接口依赖。
易错点	DIP 不是“低层依赖高层”，而是双方依赖抽象。

可能问法	什么是里氏替换原则 LSP？
标准答法	里氏替换原则要求子类型对象能够替换父类型对象而不破坏程序正确性。子类不能加强前置条件，不能削弱后置条件，必须维护父类不变式。
记忆点	子类前置放宽、后置加强、不变式维持。
易错点	几何上的 is-a 不等于行为上的可替换，例如正方形不适合继承可变长宽的矩形。

可能问法	什么是接口隔离原则 ISP？
标准答法	接口隔离原则要求客户端不应被迫依赖它不使用的方法。应把肥接口拆成多个小接口，让不同客户端只依赖自己需要的能力。
记忆点	小接口，按需依赖。
易错点	Robot 不应为了实现 Worker 被迫实现 eat/sleep。

可能问法	什么是单一职责原则 SRP？
标准答法	单一职责原则要求一个类只有一个引起变化的原因。若一个类同时承担业务计算、持久化、报表、界面等职责，任何一类变化都会影响它，应拆分为多个职责清晰的类。
记忆点	一个类，一个变化原因。
易错点	SRP 判断关键不是“只做一个方法”，而是“变化原因是否单一”。

可能问法	什么是迪米特法则？
标准答法	迪米特法则又称最少知识原则，要求对象只与直接朋友通信，不依赖间接对象的内部结构。它可减少消息链和访问耦合，例如用 <code>customer.getCityName()</code> 代替 <code>customer.getAddress().getCity()</code> 。
记忆点	只和直接朋友说话。
易错点	长链式调用通常暴露内部结构。

可能问法	为什么组合优于继承？
标准答法	继承是白盒复用，子类依赖父类实现细节，父类变化可能影响所有子类；组合是黑盒复用，只依赖对象接口，运行时可替换，职责分离更清晰。因此优先用组合，只有满足稳定行为替换关系时才使用继承。
记忆点	继承强耦合，组合更灵活。
易错点	不是禁止继承，而是谨慎使用继承。

可能问法	什么是策略模式？
标准答法	策略模式定义一组算法族，将它们分别封装为实现同一接口的策略类，使算法可以相互替换，并让算法变化独立于使用算法的客户。它常用于消除根据类型选择算法的 switch/if。
记忆点	封装算法族，用组合替代分支。
易错点	策略模式关注行为变化，不是对象创建问题。

六、构造、测试与质量

可能问法	什么是软件构造？
标准答法	软件构造是通过编码、验证、单元测试、集成、调试、构建管理和重构等活动，把详细设计转化为可运行软件的过程。它是设计的延续，而不只是写代码。
记忆点	编码 + 验证 + 测试 + 集成 + 调试 + 重构。
易错点	不会考框架注解细节，但会考构造活动和质量思想。

可能问法	什么是重构？
标准答法	重构是在不改变软件外部可观察行为的前提下，改进内部结构的内部活动。目的包括提高可读性、降低复杂度、消除坏味道、改善可维护性和可扩展性。
记忆点	外部行为不变，内部结构变好。
易错点	重构前后应通过测试保证行为一致。

可能问法	什么是代码坏味道？
标准答法	代码坏味道是代码中暗示设计或实现可能存在问题的症状，如过长方法、过大类、重复代码、魔法数字、过多参数、多处相似控制结构、过多注释掩盖逻辑不清等。
记忆点	坏味道是症状，不一定立即是错误。
易错点	发现坏味道后要说明重构方向，如提取方法、拆分类、引入策略。

可能问法	什么是防御式编程？
标准答法	防御式编程是在方法与外界交互时，不能假设外部输入、系统环境或依赖对象总是正确，因此通过参数校验、异常处理、断言和边界检查提高程序健壮性。
记忆点	不信任外部，保护内部。
易错点	异常处理外部可预期错误；断言检查内部不应违反的条件。

可能问法	什么是圈复杂度？
标准答法	圈复杂度衡量程序控制流复杂程度，也可理解为独立路径数量。常用公式 $V(G)=E-N+2$ ，实践中可近似为判定节点数+1。复杂度越高，理解和测试成本越高。
记忆点	每多一个分支判断，通常复杂度加 1。
易错点	顺序代码复杂度为 1；if/while/for/case 等会增加复杂度。

可能问法	什么是黑盒测试？
标准答法	黑盒测试不关注代码内部结构，而是根据需求、规格、用例和输入输出关系设计测试，验证系统功能是否符合预期。常见方法包括等价类划分、边界值分析、决策表、状态转换和基于用例的测试。
记忆点	不看代码，测功能。
易错点	黑盒不是随便点点，而是依据规格系统选取测试用例。

可能问法	什么是白盒测试？
标准答法	白盒测试关注代码内部结构和控制流，根据语句、分支、条件、路径等覆盖标准设计测试，验证程序内部逻辑是否被充分执行。
记忆点	看代码，测路径。
易错点	覆盖率高不代表一定没有缺陷，只代表执行充分性更高。

可能问法	单元测试、集成测试、系统测试有什么区别？
标准答法	单元测试关注单个类或方法，常用驱动和桩隔离依赖；集成测试关注模块或类之间协作是否正确；系统测试关注整个系统是否满足需求，通常以需求规格和用例文档为依据。
记忆点	单元测小，集成测接口，系统测需求。
易错点	系统测试更关注不符合需求的缺陷，而不仅是代码错误。

可能问法	什么是桩程序和驱动程序？
标准答法	桩程序模拟被测部件调用的下层或外部依赖，为被测部件提供响应；驱动程序负责创建被测部件的执行环境，调用被测部件并检查测试结果。
记忆点	桩替下游，驱动调被测。
易错点	自顶向下集成常用桩，自底向上集成常用驱动。

可能问法	等价类划分和边界值分析是什么？
标准答法	等价类划分把输入域划分为若干有效和无效类别，从每类中选代表值测试；边界值分析关注输入或输出边界附近的值，因为错误常发生在边界。二者都是常用黑盒测试方法。
记忆点	等价类选代表，边界值盯边缘。
易错点	边界常选 min、min+1、max-1、max 及越界值。

可能问法	语句覆盖、分支覆盖和路径覆盖有什么区别？
标准答法	语句覆盖要求每条语句至少执行一次；分支覆盖要求每个判定的真假分支至少执行一次；路径覆盖要求覆盖程序中的独立执行路径，强度更高但成本也更大。
记忆点	语句 < 分支 < 路径，强度逐渐提高。
易错点	语句覆盖可能漏掉某些分支错误。

七、人机交互、AI 与综合概念

可能问法	什么是人机交互设计？
标准答法	人机交互设计关注用户如何通过界面与系统完成任务，贯穿需求开发和软件设计全过程。它包括理解用户和任务、设计交互方式、原型化、评估和迭代改进。
记忆点	以用户为中心，不是最后美化界面。
易错点	HCI 题答原则和理由即可，不必死背标准原文。
可能问法	Pressman 三条黄金原则是什么？
标准答法	三条黄金原则包括：让用户掌控系统，减少用户记忆负担，保持界面一致性。它们分别强调可控、可理解和可预测的交互体验。
记忆点	掌控、少记、一致。
易错点	答界面好坏时要结合具体问题说明。
可能问法	Nielsen 可用性启发原则可如何记忆？
标准答法	常用原则包括系统状态可见、系统与现实匹配、用户控制与自由、一致性、错误预防、识别而非回忆、灵活高效、美观简洁、帮助识别并恢复错误、帮助文档。
记忆点	可见、一致、防错、可恢复、少记忆。
易错点	考试可不逐字背，指出问题并给出合理原则解释即可。
可能问法	AI 辅助软件工程时，工程师为什么仍重要？
标准答法	AI 可以辅助生成需求、UML 图、代码和测试，但生成结果可能存在层次错误、关系错误、遗漏边界条件、违反设计原则等问题。工程师需要掌握软件工程概念，才能审查、修正和决策。
记忆点	AI 出初稿，人负责判断。
易错点	不要只问 AI 得结果而不判断好坏。
可能问法	为什么 UML 图贯穿软件生命周期？
标准答法	UML 图是对软件不同阶段产物的可视化表征。需求阶段可用用例图、系统顺序图、概念类图；体系结构阶段可用包图、组件图、部署图；详细设计阶段可用类图、顺序图、状态图；构造前可用流程图或伪代码表达方法逻辑。
记忆点	不同阶段，不同图表达不同关注点。
易错点	图不是为了画图而画图，而是帮助沟通、分析和验证。

八、考前背诵压缩版

概念	一句话答案
软件工程	用系统化、规范化、可度量的方法开发、运行和维护软件。
生命周期	软件从提出到退役的全过程；模型是阶段组织方式。
持续集成	频繁合并代码并自动构建测试，尽早发现集成问题。
需求工程	获取、分析、建模、规格说明、验证和管理需求的活动。
三层需求	业务需求讲为什么，用户需求讲用户目标，系统级需求讲系统交互规格。
用例	参与者为达成业务目标与系统交互的一组场景。

概念	一句话答案
概念类图	分析阶段的问题域静态模型，有类、属性、关联、多重性，无方法。
系统顺序图	把系统当黑盒，画 Actor 与 System 的消息顺序。
体系结构	系统高层构件、连接、接口、约束和关键设计决策。
分层	按展示、业务逻辑、数据访问等职责分层，上层通过接口使用下层。
MVC	Model 管业务数据，View 管展示，Controller 管输入协调。
PO/VO	PO 面向持久化，VO 面向展示数据传递。
DAO	封装数据访问逻辑的数据访问对象。
GRASP	指导面向对象职责分配的原则集合。
SOLID	SRP、OCP、LSP、ISP、DIP 五个类设计原则。
信息隐藏	每个模块隐藏可能变化的设计决策，只暴露稳定接口。
重构	外部行为不变，内部结构改善。
黑盒测试	不看代码，根据需求规格测功能。
白盒测试	看代码，根据控制流测路径和覆盖。
HCI	围绕用户任务设计、原型化、评估和改进交互体验。

简答题万能结构：先定义，再说明作用，然后给一个例子或区别。比如问“DAO 是什么”：定义 Data Access Object；作用是封装数据访问、隔离业务层和数据库；例子是 BookDao.findById；区别是 DAO 不是数据库表，而是数据访问层对象/接口。